

GitHub Copilot Hands-On Training

Advanced BMS Diagnostics Use Cases

This syllabus outlines three specialized, industry-relevant diagnostic use cases for a Building Management System (BMS) Software Engineer. Each scenario is structured around the full SDLC: requirements analysis, design, development, unit testing, code review, CI/CD pipeline automation, and AWS deployment.

Target Audience:	BMS Software Engineers & Systems Architects
Core Focus:	Full SDLC Automation, Edge Diagnostics, AWS Cloud Integration
Technologies:	Python, C++, Terraform, GitHub Actions, AWS IoT Core, DynamoDB, S3

Use Case 4: Sensor Plausibility & Cross-Calibration Diagnostic Engine

1. Requirement Analysis

Context: In a BMS environment, sensors can experience "frozen" failures (returning a static reading despite environmental changes) or drift out of calibration over time. This diagnostic service runs continuously to identify anomalous sensor behavior by cross-referencing physically adjacent sensors.

- Ingest real-time readings from three adjacent temperature sensors (e.g., Zone A_1, Zone A_2, Zone A_3).
- Implement a sliding window to analyze variation. If a sensor's standard deviation over 1 hour is less than 0.05°C (while neighbors fluctuate), flag it as "Frozen".
- If a sensor's reading deviates from the median of its neighbors by more than 1.5°C for more than 110ms, flag it as "Out of Spec" (OOS).
- Expose a diagnostic reset API to clear calibration offsets back to zero once repairs are confirmed.

2. Design

Architecture: Python-based edge-running diagnostic engine communicating state via JSON payloads.

Copilot Prompt: Design a Python data structure to keep track of a rolling window of temperature values for multiple BMS sensor IDs.

Copilot Prompt: Create a flowchart explaining a cross-calibration voting logic schema where three inputs are compared to identify a single faulty sensor.

3. Develop

Implementation: Write the core plausibility engine with rolling statistic calculations.

Copilot Prompt: Write a Python class named `BMSDiagnosticsEngine` that compares three temperature readings, flags any sensor whose value deviates by $>1.5^{\circ}\text{C}$ for more than 110ms as faulty, and resets long-term calibration offsets to zero upon repair.

Copilot Prompt: Optimize the rolling variance algorithm in Python using numpy for memory efficiency in an embedded environment.

4. Test & Review

Testing Strategy: Unit test with mock data representing sensor spikes, gradual drift, and absolute freezing.

Copilot Prompt: Write pytest unit tests using pytest-mock to simulate one frozen sensor, one drifting sensor, and three healthy sensors, asserting the correct status code is assigned to each.

5. CI/CD Pipeline

Automation: Set up GitHub Actions to trigger tests, build a slim Docker container, and run a security vulnerability scan (using Trivy or Anchore) on the container filesystem.

Copilot Prompt: Write a GitHub Actions CI workflow that runs unit tests, builds a Docker image, runs a Trivy vulnerability scan, and fails the build if any High or Critical vulnerabilities are detected.

6. AWS Deployment

Infrastructure: Push sensor faults from the edge to the cloud. Deploy AWS IoT Core rules that route flagged anomalies to Amazon DynamoDB and trigger alerts.

Copilot Prompt: Create a Terraform script to set up an AWS IoT Core Topic Rule, routing diagnostic fault events to a DynamoDB table named BmsSensorFaults.

Use Case 5: Diagnostic Communication Manager (DCM) UDS Protocol Adapter

1. Requirement Analysis

Context: For field technicians using handheld devices, the BMS must expose low-level diagnostics. This use case models a Diagnostic Communication Manager (DCM) that adapts Unified Diagnostic Services (UDS) concepts (ISO 14229) to translate diagnostic commands over BACnet, Modbus, or CAN.

- Parse incoming diagnostic hex buffers.
- Support Service Identifier (SID) 0x19 (Read Diagnostic Trouble Codes - DTCs).
- Support Service Identifier 0x14 (Clear Diagnostic Information).
- Translate active/inactive status bytes to output positive response hex frames or negative response codes (NRCs) such as 0x12 (SubFunctionNotSupported).

2. Design

Architecture: High-performance, low-latency C++ dispatcher or micro-Python wrapper.

Copilot Prompt: Design a sequence diagram showing a handheld diagnostic scanner sending a UDS packet over serial, the DCM adapter decoding the frame, retrieving active DTCs from memory, and responding with a structured hex packet.

3. Develop

Implementation: Create a request/response dispatcher parsing command bytes.

Copilot Prompt: Write a C++ class that parses incoming UDS diagnostic requests (SID 0x19 to read active/inactive DTCs and SID 0x14 to clear DTCs) and outputs a structured hex byte response.

Copilot Prompt: Implement a binary masking utility in C++ to extract individual status bits (warning active, pending fault, historical fault) from a single status byte.

4. Test & Review

Testing Strategy: Unit testing covering positive execution and negative responses (e.g., querying unsupported services).

Copilot Prompt: Write a suite of unit tests in C++ (using Google Test) that passes raw hex array requests to the DCM and validates the exact structure of the returned byte array, including handling invalid SIDs with a Negative Response Frame.

5. CI/CD Pipeline

Automation: Compile C++ code with multiple compiler flags (GCC, Clang) to ensure cross-platform compatibility for embedded deployment.

Copilot Prompt: *Generate a GitHub Actions workflow to compile a C++ project using CMake, running both GCC and Clang compilers, and executing ctest to run the test suite.*

6. AWS Deployment

Infrastructure: Package the diagnostic schemas and log parsers in AWS. Expose a diagnostic REST API via API Gateway and AWS Lambda to allow remote web-dashboards to query the status of active DTCs.

Copilot Prompt: *Generate a Serverless Framework configuration yaml to deploy an AWS Lambda function running our diagnostic decoder with an API Gateway endpoint /diagnostics/dtc.*

Use Case 6: Persistent Data Memory (PDM) Fail-Safe Writer for Error Logs

1. Requirement Analysis

Context: Critical safety alarms (e.g., high-voltage electrical interlock failure, boiler overheat, gas leak detection) must be logged immediately. If power fails or the controller resets, these logs must be written to non-volatile memory (NVM/Flash) reliably without corrupting the file system.

- Maintain a circular ring buffer in local storage.
- Ensure thread-safe writes to prevent write collisions.
- Append a Cyclic Redundancy Check (CRC-32) hash to every log transaction.
- Upon network recovery, sync cached local diagnostic files to cloud-based cold storage (S3) and clear local memory.

2. Design

Architecture: Thread-safe file logger written in Python or Rust with local disk fallback.

Copilot Prompt: Create a system design document outline describing a local-first fail-safe logging microservice that caches data locally during network outages and synchronizes with Amazon S3 using multi-part uploads upon reconnection.

3. Develop

Implementation: Implement the file writing routine with atomic operations and CRC verification.

Copilot Prompt: Create a Python implementation of a ring-buffer error logger that performs a CRC check on every transaction, handles disk-write timeout errors, and triggers an AWS Lambda to backup logs when online.

Copilot Prompt: Write a thread-safe file writer in Python using threading.Lock to write structured error logs to a local file with atomic rename operations to prevent corruption.

4. Test & Review

Testing Strategy: Test write operations under simulated lock contention and abrupt shutdowns.

Copilot Prompt: Write unit tests in pytest that simulate a full disk exception, verifying that the logger drops lowest-priority logs to preserve high-severity alarm logs.

5. CI/CD Pipeline

Automation: Build a release pipeline that auto-versions diagnostic builds and uploads compiled release binaries as GitHub Release assets.

Copilot Prompt: Create a GitHub Actions workflow that runs on a git tag push (e.g., v1.2.0), packages the application, runs tests, and creates a GitHub Release with the compiled binary attached.

6. AWS Deployment

Infrastructure: Set up a secure Amazon S3 Bucket with Lifecycles to transition error logs from Active to Glacier Deep Archive, along with IAM policies ensuring the edge device has Write-Only permissions.

Copilot Prompt: Write a Terraform file defining an Amazon S3 Bucket with a lifecycle policy to transition logs to Glacier after 30 days, and create an IAM User with a policy restricted to s3:PutObject on that bucket.

Specialized Copilot Diagnostic Training Tips

1. Translating Protocols

Use Copilot Chat to understand low-level byte arrays.

Copilot Prompt: Analyze this array of bytes: [0x02, 0x19, 0x02, 0x0D, 0x00]. In the context of UDS (ISO 14229), what diagnostic service and sub-function is being requested?

2. Generating Boundary Conditions

When prompting Copilot for tests, explicitly ask it to target the physics of the physical system.

Copilot Prompt: Generate edge-case test values for a room temperature sensor. Include values like absolute zero, sudden step changes of 50 degrees in 1 millisecond, and NaN values, asserting how our filter responds.

3. Refactoring Monoliths

If you have large legacy diagnostic scripts, open Copilot Chat and highlight a section.

Copilot Prompt: Refactor this legacy diagnostic function into smaller, single-responsibility methods. Extract the validation logic, database write logic, and alerting logic into separate testable components.